

Smart Classroom — Multilingual Lecture Assistant

BOB HACKS '26 — Problem Statement 1 Submission Docs

1. Product Requirement Document (PRD)

1.1 Problem statement

University classrooms in India are linguistically diverse. A professor lectures in English while drawing diagrams and formulas on a whiteboard; students with varying English proficiency (more comfortable in Hindi, Bangla, Arabic, etc.) fall behind because manual note-taking and translation can't keep pace with a fast-moving technical lecture. Existing tools either translate speech with no memory of what was drawn, or stream video/whiteboard content with no language support — none combine both while staying lightweight enough for real classroom conditions (variable wifi, no dedicated hardware).

1.2 Goals

- Let a student follow a live lecture in their preferred language without losing sync between what's said and what's drawn.
- Preserve the lecture as a reviewable record (not just a live-only experience).
- Keep technical/CS vocabulary intact rather than mistranslated.
- Work reliably on standard classroom wifi, not dependent on studio-grade bandwidth.

1.3 Non-goals (for this build)

- Not attempting real-time word-for-word simultaneous interpretation (explicitly avoided — see tech-stack fixes).
- Not a video-conferencing replacement; single-classroom, single-teacher use case only.
- Not diagnosing *why* a student is confused (no learning-analytics layer in MVP).

1.4 Target users

- **Primary:** Students in multilingual classrooms who are less comfortable following lectures in English.
- **Secondary:** Professors who want their lecture made accessible without changing how they teach (no new tools to learn beyond a drawing pad + mic).

1.5 Success metrics (for the hackathon demo)

- End-to-end latency from spoken sentence to delivered translated caption/audio: target under ~4 seconds (segment-buffered, not truly real-time).
- Zero loss of whiteboard content — every stroke persisted and replayable.
- Technical terms (e.g. "recursion", "polymorphism") preserved untranslated across at least one full demo lecture segment.
- Judges can toggle language mid-demo and see content update within the same session without reload.

1.6 Constraints & assumptions

- Hackathon timeframe — architecture must be buildable by a small team in the available hours.
 - Classroom wifi assumed available but not guaranteed high-bandwidth — drives the "vector strokes, not video" decision.
 - Demo will likely use a pre-recorded or controlled lecture clip rather than a live noisy mic, to reduce demo risk.
-

2. Feature Specification

#	Feature	Description	Priority	Acceptance criteria
F1	Teacher speech capture	Mic input captured and chunked into segments (per sentence/pause) rather than streamed continuously	MVP	Segment boundaries align with natural pauses; no segment exceeds ~8s
F2	Whiteboard stroke capture	Digital pen/tablet input captured as vector stroke events (x/y/pressure/tim	MVP	Strokes reconstructable pixel-accurately from stored event log

#	Feature	Description	Priority	Acceptance criteria
		estamp), not video		
F3	Segment-to-transcript (ASR)	Each audio segment transcribed to English text	MVP	≥90% transcription accuracy on demo lecture audio
F4	Structured concept pairing	Each transcript segment paired with the stroke batch drawn in the same time window	MVP	Every transcript segment has an associated (possibly empty) stroke batch
F5	Translation with term preservation	Explanatory language translated to selected language; recognized technical/CS terms kept in English	MVP	Seeded technical-term list stays untranslated in ≥95% of occurrences
F6	Language toggle	Student can switch display language at any point without losing position in the lecture	MVP	Toggle updates captions within 1 interaction, no page reload
F7	Live caption delivery	Translated captions delivered to student view in near-real time (buffered)	MVP	New segment appears within ~4s of teacher finishing that sentence

#	Feature	Description	Priority	Acceptance criteria
F8	Optional TTS playback	Student can toggle audio narration of the translated captions	Should-have	Toggle on/off without breaking caption sync
F9	Session persistence	Full transcript + stroke history stored per session	MVP	Session fully reloadable after refresh/reconnect
F10	Replay/scrub	Student can scroll back through earlier segments and re-view associated strokes	Should-have	Clicking an earlier caption jumps the whiteboard replay to that point
F11	Multi-language support (beyond one)	Support for Hindi + at least one more (Bangla/Arabic)	Nice-to-have	Second language demoable, may have lower polish
F12	Low-connectivity fallback	Local/on-device ASR fallback when classroom wifi is poor	Stretch (post-hackathon)	Documented as future work if not built

3. User Flows

3.1 Teacher flow

1. Teacher opens the teacher app, starts a new lecture session.
2. Teacher speaks naturally and draws on the digital whiteboard/pad as usual.

3. App captures mic audio (chunked) and pen strokes (vector events) in the background — no extra action needed from the teacher.
4. Teacher ends the session when the lecture is over; session is finalized and stored.

3.2 Student flow

1. Student opens the student web app, joins the active session (or opens a past one).
 2. Student selects preferred language from a dropdown/toggle.
 3. As the teacher speaks and draws, the student sees:
 - The whiteboard replaying strokes in near-real time.
 - Translated captions appearing below/beside the board, technical terms shown in English.
 4. Student may toggle TTS to hear the captions read aloud.
 5. Student can scroll back to any earlier point; whiteboard view jumps to match.
 6. After class, student reopens the same session link to review the full lecture at their own pace.
-

4. Flowcharts

4.1 High-level system flow (Mermaid)

flowchart TD

A[Teacher speaks + draws] --> B[Speech capture: chunked ASR]

A --> C[Whiteboard capture: vector strokes]

B --> D[Sync and translation engine]

C --> D

D --> E[Session store]

E --> F[Student web app]

F -->|language toggle| D

4.2 Segment processing flow (Mermaid)

flowchart TD

S1[Audio chunk ends at natural pause] --> S2[ASR transcribes chunk to English]

S2 --> S3[Pair chunk with stroke batch in same time window]

S3 --> S4[Identify technical terms to preserve]

S4 --> S5[Translate explanatory text to target language]

S5 --> S6[Attach timestamp, store segment]

S6 --> S7[Push segment to connected student sessions]

4.3 Student session flow (Mermaid)

flowchart TD

U1[Student opens session link] --> U2[Select language]

U2 --> U3{Live or past session?}

U3 -->|Live| U4[Subscribe to live segment stream]

U3 -->|Past| U5[Load stored segments from session store]

U4 --> U6[Render captions + replay strokes]

U5 --> U6

U6 --> U7{Student scrolls back?}

U7 -->|Yes| U8[Jump whiteboard replay to selected segment]

U7 -->|No| U6

5. UI/UX Specification

5.1 Teacher app

- **Screen: Session control**
 - Large "Start lecture" / "End lecture" button (primary action, minimal chrome — teacher shouldn't be distracted mid-lecture).
 - Small unobtrusive status indicator: mic active, pen active, session ID.
 - No language selection needed here — teacher always speaks/writes naturally in their own language.
- **Screen: Whiteboard canvas**
 - Standard drawing canvas (pen, eraser, color, clear) — should feel like any digital whiteboard app, no added complexity.
 - Background capture indicator (subtle, e.g. a small pulsing dot) confirming strokes/audio are being recorded.

5.2 Student web app

- **Screen: Join / select session**
 - Simple list or link-based entry to a session (live sessions marked "Live now", past ones show date/title).
- **Screen: Lecture view (primary screen)**
 - **Top bar:** language dropdown (persistent, always visible), TTS on/off toggle, live/past indicator.
 - **Main area, split view:**
 - Left/top: whiteboard replay panel — strokes render progressively, matching pacing of the original drawing.
 - Right/bottom: caption feed — scrolling list of translated segments, each with a timestamp; current/live segment highlighted.
 - Technical terms in captions rendered in a distinct style (e.g. monospace or subtly bolded) so students visually recognize "this stayed in English on purpose," not a translation gap.
 - **Scrub bar:** timeline at the bottom of the whiteboard panel; clicking any point jumps both panels to that moment.
- **States to design for:**
 - Loading/connecting to a live session (skeleton loader, not blank screen).
 - Reconnect after network drop (auto-resume from last received segment, visible "reconnecting..." indicator).
 - Empty state for a session with no content yet (teacher hasn't started).

- Language-switch transition (captions re-render instantly without disrupting whiteboard playback position).

5.3 Design principles

- **Low cognitive load:** captions and board share visual weight — neither should force the student to choose between "read" and "watch."
- **Trust signals:** visibly distinguishing preserved technical terms builds trust that the system isn't guessing on jargon.
- **Resilience over polish:** given classroom wifi variability, prioritize clear loading/reconnect states over animation flourish.
- **Accessibility:** caption text sizing adjustable; TTS as an alternative channel for students who prefer audio over reading.